

GYWS Leave & Attendance Management System

System Design, Architecture, and Database Schema Report

Project Members:

Sharanya Baidya

Hostel Committee Officer

Khushi Srivastava

Human Resource Manager

Aaratrika Bhattacharya

School Development Officer

Gopali Youth Welfare Society

June 2026

Contents

1	Introduction and Project Overview	3
1.1	Organizational Context and Project Scope	3
1.2	Problem Statement	3
1.3	Proposed Solution	3
1.4	Unified Execution Thread: Leave Rules, Attendance, and Payroll	4
1.5	Process Automation, Transparency, and Governance	4
2	System Design and Class Architecture	6
2.1	Object-Oriented Domain Models	6
2.2	Data Access via Sheet Repository Pattern	6
2.3	Strategy Pattern for Leave Type Policies	7
3	Architectural Design and Platform Integration	8
3.1	Client-Server Communication Architecture	8
3.2	Apps Script RPC mechanism	8
3.3	Spreadsheet Integration Layer	8
3.4	Live Deployment and Usage	9
4	Database Design and Entity Relationship Schema	9
4.1	Entity Relationship Diagram (ERD)	9
4.2	Relational Schema Tables	9
5	Conclusion and Future Scope	13
5.1	Conclusion	13
5.2	Future Scope	13

1 Introduction and Project Overview

1.1 Organizational Context and Project Scope

Gopali Youth Welfare Society (GYWS) is a non-governmental organization dedicated to social upliftment, education, and community development. Managing administrative workflows—such as tracking volunteer and staff attendance—requires a robust, reliable, and secure information system.

The **GYWS Leave & Attendance Management System** was developed as a unified administrative solution to automate, digitize, and streamline:

- **Human Resource Operations:** Supervised by the Human Resource Manager, the system provides a unified portal for employees to submit applications, track attendance histories, and verify balances, while enabling the HR department to oversee the employee record lifecycle, approve or reject leaves, track yearly leave entitlements, and generate monthly salary calculations.

1.2 Problem Statement

Prior to this system’s implementation, leave tracking and attendance registers were managed manually through physical logbooks, disconnected spreadsheets, and manually composed emails. This manual workflow suffered from several distinct vulnerabilities:

1. **Data Redundancy and Discrepancies:** Discrepancies between physical attendance logs and payroll calculations led to discrepancies in salary calculation.
2. **High Latency in Approval Loops:** Employees had to wait days for managers to review paper forms, slowing down operations.
3. **Difficult Maintenance and Oversight:** The lack of a centralized log meant that the leave-to-approval notification thread had to be manually tracked and checked, making it extremely tedious to monitor and maintain for coordinators.
4. **Security Vulnerabilities:** Clear-text passwords and unrestricted access to spreadsheets posed data integrity and privacy risks.
5. **Serialization Limits:** Standard script functions suffered from execution timeouts and serialization freezes when communicating between the client browser and the spreadsheet database.

1.3 Proposed Solution

The system replaces this manual approach with a secure, serverless Web Application. The application is built on top of **Google Apps Script (GAS)** and **Google Sheets**, which eliminates hosting costs while ensuring institutional data is stored securely in the organization’s Google Drive.

The application architecture transitions the codebase to a strict **Object-Oriented Programming (OOP)** model, using:

- **Encapsulated Domain Models** to govern employee properties and leave limits.
- **Sheet Repository Pattern** to decouple physical database columns from application code and implement in-memory query caching.
- **Strategy Pattern** to modularize notice periods and quota validation rules for each distinct type of leave (Casual, Medical, Emergency, and Half-Day).

1.4 Unified Execution Thread: Leave Rules, Attendance, and Payroll

To ensure administrative consistency, the system unifies employee portals, leave rule strategy validation, attendance logs, and payroll calculations into a single execution thread. Figure 1 conceptualizes this systematic workflow:

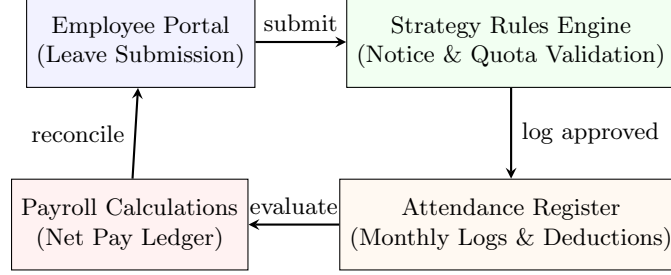


Figure 1: Systematic Integration Thread

The integration thread operates as follows:

1. **Strategy Validation:** When an employee applies for leave (Casual, Medical, Emergency, or Half-Day) via the portal, the request triggers the corresponding `LeaveTypeStrategy` to verify notice requirements and verify that yearly entitlement limits are not exceeded.
2. **Attendance Integration:** Upon approval by the supervisor, the database appends the record, and the system automatically recalculates the employee's monthly attendance summary (summing days present, approved leave types, and unapproved absent days).
3. **Payroll Reconciliation:** During monthly payroll runs, the salary calculation engine parses the attendance summaries. For any days absent or leaves taken in excess of the annual balance limit (`ExtraDays`), a proportional deduction is calculated using the employee's daily pay rate. The final output is written as a unified transactional entry in the `SalaryCalculations` ledger.

1.5 Process Automation, Transparency, and Governance

The redesigned system addresses these operational bottlenecks by introducing a high level of process automation, visibility, and robust governance tools:

- **Automated Notification Loops:** The manual coordination of application status notifications has been completely automated. When an employee submits a leave request, the system automatically dispatches structured HTML email alerts to the designated approving manager. Upon approval or rejection, another automated email notification is instantly sent back to the employee, closing the communication loop without any administrative intervention.
- **Enhanced Transparency for Employees:** All staff members gain direct, read-only visibility into their historical leave data, monthly attendance registers, and remaining entitlements. This self-service transparency resolves scheduling conflicts and eliminates administrative back-and-forth.
- **Governance and Super-Admin Control:** Governing Body Members are registered in the system as `super_admin` users. This role provides them with administrative dashboards to oversee operational trends, view detailed audit logs of all system transactions, and modify configuration records dynamically, ensuring complete transparency and compliance.

This documentation details the system design, communication protocols, entity relationships, and relational schemas that govern the live application.

2 System Design and Class Architecture

2.1 Object-Oriented Domain Models

To improve maintainability, reduce coupling, and eliminate procedural anomalies, the backend database tables are mapped to encapsulated JavaScript classes. The primary domain models are described below:

Table 1: Core Domain Entities and Attributes

Class	Attributes	Encapsulated Business Behaviors
Employee	id, name, email, department, role, salary, isActive	Calculates daily pay rate via <code>getDailyRate()</code> and fetches role-based leave limits via <code>getMaxAnnualLeaves()</code> .
LeaveApplication	appId, employeeId, leaveType, fromDate, toDate, totalDays, status	Evaluates application parameters and provides dynamic row formatting helpers.
AttendanceRecord	attId, employeeId, month, year, workingDays, daysPresent	Tracks monthly records and calculates absent days dynamically.
LeaveBalanceRecord	employeeId, year, totalEntitled, approvedLeaveUsed, medicalLeaveUsed	Tracks YTD quotas, calculating remaining balances and entitlement surpluses.

2.2 Data Access via Sheet Repository Pattern

The database interaction layer uses a generic `SheetRepository` class which decouples the domain entities from the physical columns of the Google Spreadsheet.

```
1 class SheetRepository {
2   constructor(sheetName) {
3     this.sheetName = sheetName;
4     this.sheet = SpreadsheetApp.getActiveSpreadsheet().getSheetByName(
5       sheetName);
6     this.headers = [];
7     this.normalizedHeaders = [];
8     this.records = []; // Array of { item: normalizedObj, rowIndex:
9       number }
10    this.load();
11  }
12  load() {
13    const range = this.sheet.getDataRange();
14    const values = range.getValues();
15    if (values.length > 0) {
16      this.headers = values[0].map(h => String(h).trim());
17      this.normalizedHeaders = this.headers.map(h => this.normalizeKey(
18        h));
19      this.records = values.slice(1).map((row, index) => {
20        const data = {};
21        this.normalizedHeaders.forEach((normHeader, colIndex) => {
22          data[normHeader] = row[colIndex];
23        });
24      });
25    }
26  }
27 }
```

```

21         return { item: data, rowIndex: index + 2 };
22     });
23 }
24 }
25 }

```

Listing 1: SheetRepository Class Design

2.3 Strategy Pattern for Leave Type Policies

Rules for verifying notice periods, balance checks, and salary deductions are governed by the Strategy Pattern. Adding a new leave type does not require changes to core submission logic; it only requires registering a new class.

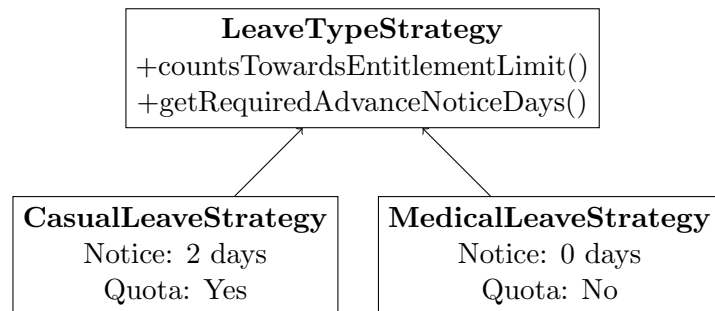


Figure 2: UML Strategy Pattern for Leave Rules

3 Architectural Design and Platform Integration

3.1 Client-Server Communication Architecture

The system is constructed as a Serverless Web Application running entirely within the Google Cloud sandbox. Figure 3 illustrates the system components:

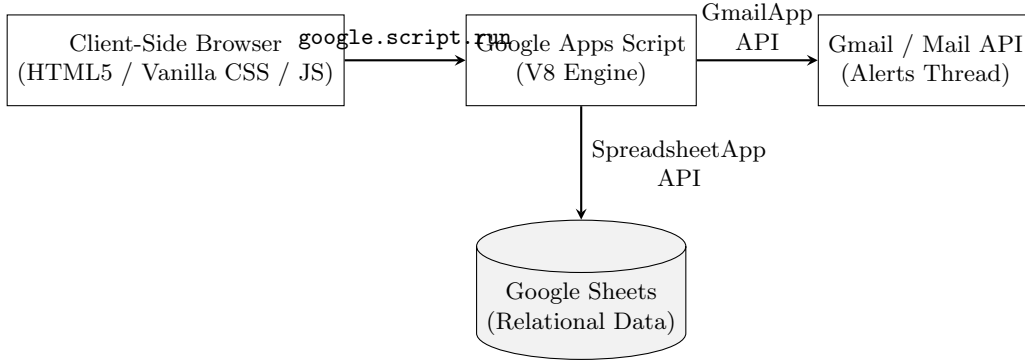


Figure 3: System Architecture Flowchart

3.2 Apps Script RPC mechanism

The client-side scripts communicate asynchronously with the backend using the remote procedure call (RPC) mechanism provided by the `google.script.run` API.

- **Request Initiation:** Client-side JavaScript makes asynchronous calls using callback registrations:

```
1  google.script.run
2    .withSuccessHandler(data => renderDashboard(data))
3    .withFailureHandler(err => showToast(err.message, 'error'))
4    .getDashboardData(userId, role);
5
```

- **Serialization constraint:** Backend responses must be converted to plain JavaScript objects. We resolve this constraint by deep-serializing the class instances via a `toPlainObject()` converter, returning clean, prototype-free key-value dictionaries.

3.3 Spreadsheet Integration Layer

Database interactions are performed using Google's Google Sheets API service (`SpreadsheetApp`).

1. **Batch Loading:** Instead of making cell-by-cell queries, the `SheetRepository` fetches the entire data range via `sheet.getDataRange().getValues()` in a single execution. This operates on a 2D array in RAM, optimizing throughput.
2. **Normalized Matching:** The repository maps query criteria dynamically, casting comparisons to uniform strings to resolve database type mismatches (e.g. string inputs vs numerical spreadsheet values).
3. **Concurrent Operations:** Cell updates are written back using selective cell bindings, followed by `SpreadsheetApp.flush()` to ensure database updates commit immediately before notifications are dispatched.

3.4 Live Deployment and Usage

This project is live and currently used as the primary HR administrative tool. The serverless architecture eliminates infrastructure costs and provides seamless security via Google's OAuth authentication mechanisms, locking access down according to roles while sending automated email confirmation loops through **GmailApp** to the managers.

4 Database Design and Entity Relationship Schema

The database layer of the system uses Google Sheets as a relational database, where individual sheet tabs represent relations (tables) and columns represent attributes. Relationships are maintained logically using primary and foreign key columns.

4.1 Entity Relationship Diagram (ERD)

The diagram below, generated using the **tikz** graphics library, illustrates the relationships between the core database entities.

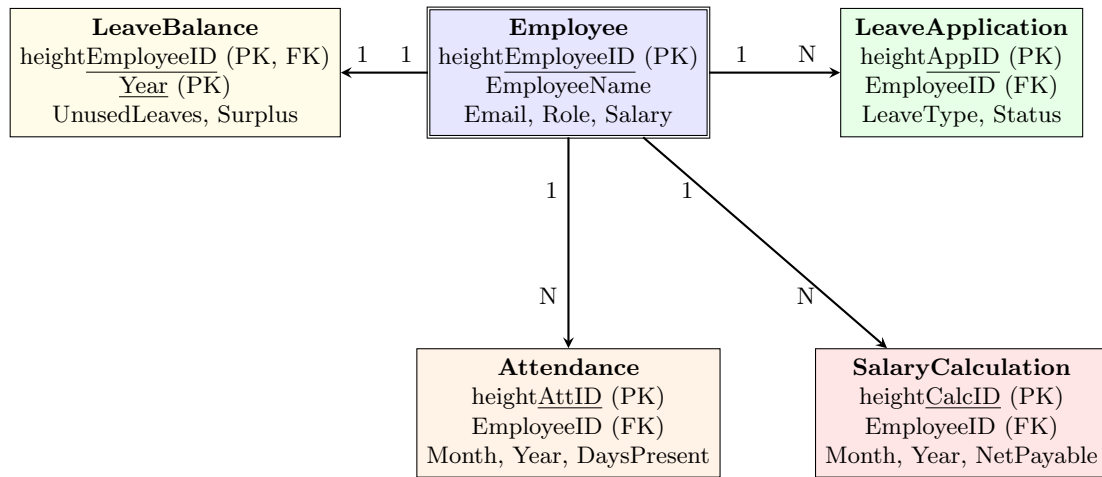


Figure 4: System Entity Relationship Diagram (ERD)

4.2 Relational Schema Tables

Below are the detailed column-level schemas for each table in the database:

Table 2: Employees Table Schema

Field Name	Data Type	Key	Description
EmployeeID	String	PK	Unique identifier for each employee (e.g. EMP001).
EmployeeName	String	-	Full legal name of the employee.
Email	String	-	Official email address.
Department	String	-	Department allocation (e.g. Engineering).
Role	String	-	System role: <code>employee</code> , <code>office_staff</code> , <code>hr</code> , <code>super_admin</code> .
Salary	Numeric	-	Base monthly salary in currency.
PasswordHash	String	-	SHA-256 base64 security password hash.
JoiningDate	Date	-	Target employment start date.
IsActive	Boolean	-	TRUE if employee is active; FALSE if deactivated.
Phone	String	-	Contact number.

Table 3: LeaveApplications Table Schema

Field Name	Data Type	Key	Description
AppID	String	PK	Unique identifier for the application (e.g. APP001).
EmployeeID	String	FK	Refers to the applying Employee.
EmployeeName	String	-	Denormalized copy of the Employee name.
AppliedBy	String	FK	Refers to the Employee ID who submitted the leave.
LeaveType	String	-	Casual Leave, Medical Leave, Emergency Leave, or Half Day.
FromDate	Date	-	Start date of the leave application.
ToDate	Date	-	End date of the leave application.
TotalDays	Numeric	-	Total duration of leaves (e.g. 1.0 or 0.5).
Reason	String	-	Justification note.
Status	String	-	State: <code>Pending</code> , <code>Approved</code> , or <code>Rejected</code> .
AppliedOn	Date	-	Date the application was registered.
ApprovedBy	String	FK	Employee ID of the approving manager.
ApprovedOn	Date	-	Timestamp of approval decision.
IsHalfDay	String	-	TRUE or FALSE indicator.
ReferenceNo	String	-	External reference tracking token (e.g. GYWS/L/101/2026).
MarkedAbsent	String	-	Absent outcome check: <code>PENDING</code> , <code>PRESENT</code> , <code>ABSENT</code> .

Table 4: Attendance Table Schema

Field Name	Data Type	Key	Description
AttID	String	PK	Unique attendance transaction ID (e.g. ATT001).
EmployeeID	String	FK	Refers to the Employee.
Month	String	-	Name of the calendar month (e.g. May).
Year	Numeric	-	Calendar year value.
WorkingDays	Numeric	-	Net working days in month (excluding holidays).
DaysPresent	Numeric	-	Physically present days.
ApprovedLeave	Numeric	-	Casual Leave days counted as present.
MedicalLeave	Numeric	-	Medical Leave days counted as present.
EmergencyLeave	Numeric	-	Emergency Leave days counted as present.
DaysAbsent	Numeric	-	Absent days calculated for deduction.

Table 5: LeaveBalance Table Schema

Field Name	Data Type	Key	Description
EmployeeID	String	PK, FK	Refers to the Employee.
EmployeeName	String	-	Denormalized copy of the Employee name.
Year	Numeric	PK	The calendar year.
TotalEntitled	Numeric	-	Total leaves entitled (default 14).
ApprovedLeaveUsed	Numeric	-	Casual leaves used in year.
MedicalLeaveUsed	Numeric	-	Medical leaves used in year.
EmergencyLeaveUsed	Numeric	-	Emergency leaves used in year.
UnapprovedAbsentUsed	Numeric	-	Unapproved absent days registered.
RejectedCasualUsed	Numeric	-	Rejected casual leaves registered.
RejectedMedOrEmgUsed	Numeric	-	Rejected medical or emergency leaves registered.
CarryForward	Numeric	-	Unused leaves carried forward from previous year.
UnusedLeaves	Numeric	-	Net unused leaves remaining.
LeaveSurplus	Numeric	-	Excess leaves used beyond entitlement.
LastUpdated	Date	-	Timestamp of last balance calculation update.

Table 6: SalaryCalculations Table Schema

Field Name	Data Type	Key	Description
CalcID	String	PK	Unique identifier for the calculation (e.g. SAL001).
EmployeeID	String	FK	Refers to the Employee.
EmployeeName	String	-	Denormalized copy of the Employee name.
Month	String	-	Month name for payroll (e.g. May).
Year	Numeric	-	Calendar year for payroll.
BaseSalary	Numeric	-	Employee base monthly salary.
ApprovedLeaveDays	Numeric	-	Approved casual leaves taken.
AbsentDays	Numeric	-	Unapproved absent days.
HalfDays	Numeric	-	Total half days (present 0.5, absent 0.5).
MedicalDays	Numeric	-	Approved medical leave days.
EmergencyDays	Numeric	-	Approved emergency leave days.
ExtraDays	Numeric	-	Leaves taken exceeding entitlement limit.
AbsentDeduction	Numeric	-	Salary deduction due to absent days.
ExtraLeaveDeduction	Numeric	-	Salary deduction due to extra leaves.
UnusedLeaveBonus	Numeric	-	Bonus calculated on unused leave.
NetPayable	Numeric	-	Final salary payout = BaseSalary - Deductions.
CalculatedOn	Date	-	Timestamp of payroll generation.

5 Conclusion and Future Scope

5.1 Conclusion

The **GYWS Leave & Attendance Management System** is now fully live and deployed, serving as the primary administrative tool for managing HR, leave tracking, and employee attendance registers. By transitioning from physical, disconnected registers to an automated web application, the organization has achieved immediate improvements in operational efficiency.

Employee attendance is systematically synchronized from the primary Google Spreadsheet ledger, ensuring that all leave decisions, attendance metrics, and payroll calculations are linked under a single database thread. The automation of approval notification loops has drastically reduced loop latencies, while role-based logins provide clear visibility for staff and robust oversight tools for Governing Body Members.

5.2 Future Scope

While the current portal successfully digitizes leave and attendance workflows, the platform can be expanded to serve as a comprehensive staff dashboard. Future areas of development include:

- **All-Staff Portal Expansion:** Transforming the application into a central staff hub by integrating volunteer task allocation, hostel duty handovers, and school scheduling registers directly into the dashboard.
- **Performance & Feedback Tracking:** Adding features to log monthly performance reviews, feedback forms, and professional development milestones for educators and hostel staff.
- **Expense & Reimbursement Logging:** Digitizing organizational claims and petty cash tracking, allowing volunteers and staff to upload bills and track approval statuses online.
- **Mobile Application Development:** Packaging the web interface into a progressive web app (PWA) or hybrid mobile app to support offline logging and push notifications.